Coding Exercise –Transactions Ingest

Background

Your team maintains a retail payments platform that consolidates card transactions from multiple source systems. The platform retrieves transaction snapshots by calling external source-system APIs. For purposes of this exercise, assume transactions are imported from three source systems: Store Uptown, Store Downtown, and Online. Transaction identifiers are unique within a source system but are not guaranteed to be unique across all source systems.

In production, an hourly import process retrieves transactions from all configured source systems. Source systems may publish corrections, updates, delayed transactions, or remove previously published transactions. The import process must reconcile incoming data, create new records, detect changes, identify revoked transactions, and maintain an audit history.

For this exercise, you do not need to implement scheduling, background services, timers, or job orchestration. Assume an external scheduler already exists and will invoke the import process every hour. Your responsibility is to implement the import functionality itself and provide a mechanism to trigger it manually. The solution should be designed so the same import logic could later be invoked by a scheduled background job without modification.

This exercise evaluates solution architecture, backend design, data modeling, transaction reconciliation, API design, maintainability, testability, and overall code quality. The frontend application is intentionally lightweight and is primarily intended to display imported transaction data.

Functional Requirements

- Build an ASP.NET 10 web application.
- Retrieve transaction snapshots from the following source systems: Store Uptown, Store Downtown, and Online.
- Source-system integrations may be mocked using interfaces and hard-coded sample data. You may fake source-system responses using static or dynamic sample data rather than implementing actual external API integrations.
- Import new transactions into a local SQLite database.
- Detect and update existing transactions when source data changes.
- Detect revoked transactions.
- Record audit history for detected changes.
- Support idempotent execution.
- Display imported transactions.

Sample Transaction JSON

```
{
  "sourceSystem": "Store Uptown",
  "transactionId": 100001,
  "cardNumber": "***1111",
  "locationCode": "UPTOWN",
  "productName": "Wireless Mouse",
  "amount": 24.99,
  "transactionTime": "2026-02-27T08:15:30Z"
},
{
  "sourceSystem": "Store Downtown",
  "transactionId": 100001,
  "cardNumber": "***0002",
  "locationCode": "DOWNTOWN",
  "productName": "Mechanical Keyboard",
  "amount": 89.99,
  "transactionTime": "2026-02-27T09:45:12Z"
},
{
  "sourceSystem": "Online",
  "transactionId": 100001,
  "cardNumber": "***4444",
  "locationCode": "ONLINE",
  "productName": "USB-C Docking Station",
  "amount": 129.95,
  "transactionTime": "2026-02-27T11:20:05Z"
}
```

The logical unique key for a transaction is (SourceSystem, TransactionId).

Data Model

Field	Type	Notes
`SourceSystem`	string (length < 20)	Identifies the upstream source system.
`TransactionId`	int	Unique only within a SourceSystem.
`CardNumber`	string (length < 20)	The last 4 digits of a card if being used. It can be empty if using other payment methods.
`LocationCode`	string (length < 20)	Store, website, or partner location identifier.
`ProductName`	string (length < 20)	Product or item description.
`Amount`	decimal(18,2)	Transaction amount.
`TransactionTime`	DateTime (UTC)	Time when the transaction occurred in the source system.

Extensions allowed: You may add additional entities or fields if helpful (for example: Status, RevisionNumber, AuditHistory, ImportBatch, SourceMetadata, etc.).

API Requirements

`POST /api/import`	initiates a transaction import operation.
`GET /api/transactions`	returns imported transactions for display in the frontend application. Additional endpoints may be added as needed.

Frontend Requirements

The frontend application should preferably be implemented using the latest Angular version. The frontend only needs to provide a simple interface for viewing imported data and does not need to be highly polished.

Required:

- Display transactions in a table or grid
- Retrieve data from the backend API
- Display Source System, Transaction ID, Location Code, Product Name, Amount, Transaction Time, and Status

Optional Enhancements:

- Sorting
- Filtering
- Pagination
- Transaction details view
- Audit history view
- Responsive design
- Manual import button

Non-Functional Requirements

ASP.NET 10, EF Core Code-First, SQLite, externalized configuration using `appsettings.json`, logging, error handling, and automated tests where appropriate.

Acceptance Criteria

- New transactions are inserted.
- Transactions with matching (`SourceSystem`, `TransactionId`) are compared and updated when field values change. Field-level changes are recorded in audit history.

- Transactions missing from a source snapshot within 24 hours of `TransactionTime` are marked as revoked. And revocation events are recorded in audit history.
- (Optional) Transactions older than 24 hours can be finalized and become immutable.
- Import processing can be manually triggered.
- The transaction import process is idempotent (running the import multiple times with identical source data should produce the same result).
- Transaction data is displayed in the frontend application.
- The front end retrieves transaction data from backend APIs.

Deliverables

1. GitHub repository containing complete application source code.
2. README file containing:
    a. Build instructions
    b. Run instructions
    c. Design decisions and assumptions
    d. Any additional information needed to evaluate the solution

AI Usage Disclosure

State whether AI tools were used. If AI assistance was used, include prompts, instructions, conventions, rules, and design discussions as markdown files in the repository. Describe any modifications made to AI-generated output.

Miscellaneous:

1. Before coding, please estimate how many hours you may need for this task.
2. Create a Git repository and commit your code there.
3. Track your time during the implementation and record the actual hours you spent.

After finishing the coding exercise, please send us an email with:

1. The link to your Git repository.
2. The numbers of the estimated hours and the actual hours. You can optionally write a short paragraph with details and explanations.
3. Other comments (Optional):
    a. The problems you came across, and how you solved them.
    b. The highlights in your code or coding practice that you want us to notice.
    c. Anything else.